Project: Splendor Card Game

Expected Effort (approximate): *40 hours per person*
Weightage: *20% of final grade*
Due Date: *2 April, THU, 11:59pm*

(Tentative) Grading Components	Marks

Code	**10**
▪ The code is modularized with multiple classes and methods. ▪ The code is clean. http://smu.sg/clean-code . ▪ The code is designed and layered to known design principles. For example, Single Responsibility Principle (SRP). ▪ The configuration parameters are externalized. Reference: https://reflectoring.io/externalize-configuration ▫ Parameters such as the number of points to win, the initial gem count for different player numbers, and card data should be loaded from files & file paths to card data files should be in the configuration file. ▪ Adherence to Java coding convention. http://www.oracle.com/technetwork/java/codeconventions-150003.pdf ▫ The code is appropriately documented. ▫ The variable, class, and method names are meaningful. ▫ The code is indented using tabs or spaces consistently. ▫ The classes are logically organized with packages.	

Application	**5**
▪ Usability of your console application. ▪ To earn a bonus point (max 1% of final grade), you can extend the application in the following way(s): ▫ Include useful game features beyond the minimum requirements and explain in your presentation why each additional feature is beneficial. ▫ A client-Server implementation allowing two players to play over a local network. ***Note:*** *If you choose this, your design must still maintain a clean separation between the Game Logic and the Network/UI layer.* ▪ *The main consideration is usability not aesthetics (i.e. have a pretty UI). A console application can be very usable too!* ▫ Nielsen's 10 heuristic evaluation https://www.nngroup.com/articles/ten-usability-heuristics/ ▪ It is fun to play.	

Presentation	**5**

▪ Time allocation:
  ▫ 10 minutes for presentation & demonstration.
  ▫ 10 minutes for Q & A.
▪ Presentation Content:
  ▫ State your team's object-oriented design. Use UML class diagrams if necessary.
  ▫ List all the open source libraries used. If you evaluated a few, state why you picked that particular library.
  ▫ How the team designed the application. Any design considerations?
  ▪ If your app has a client-server, describe how your classes are designed/organised (packages) to maintain a clean separation between the Game Logic and the Network/UI layer.
  ▫ Demonstrate the features of your client application.
▪ The presentation is well-rehearsed.
  ▫ Slides are clean and visible.
  ▫ There is no unnecessary switching of laptops/applications.
  ▫ The transition is smooth.	

Penalty	
▪ ▪ Up to 50% penalty of your total marks if any of your Java source files does not compile or run.	

Distribution of Work

- Each member of the team is expected to be familiar with all aspects of the deliverables submitted.
- Your team may be selected for an interview at the end of the term to walkthrough your team's deliverables with us. **This is to ensure that every member of your team was involved in the project**.

Peer Evaluation

- If you have any teaming issues, do highlight to your instructors early and not wait till the peer evaluation.
- You will be awarded a 2-point (of final grade) penalty if you fail to complete the peer evaluation before the deadline.
- There is one team grade for the project. However, your individual grade can be higher or much lower than the team grade based on peers' evaluation (and instructors' evaluation based on the team's interview and observation across the semester).
- We will severely penalize free-loaders, as well as team members who decide to complete the project alone.
- Guidelines
  - Please be frank when evaluating your peers (Do not lie).
  - Do the evaluation seriously.
  - You must use the comments field to explain why you think a particular individual deserves the score. Note that the system will not force you to add comments, but we require it. You need to input comments about every member in your team for every metric. Be frank and honest with your comments.
  - Your team peer evaluation score will not be compared against other teams' scores. Having a lower team score than another team will not affect your final grade.
- You are expected to evaluate yourself and your team members based on 4 criteria on a scale of 1 – 25.
  - Commitment
  - Working with others
  - Quantity of work
  - Quality of work

The grading rubric is:

	Evaluation Scale (1 – 25)			
	1 – 8	**9 - 15**	**16 - 21**	**22 - 25**
Commitment	Often missed meetings (>75% of total meetings).	Missed one or two meetings without reasons and failed to inform members about absence prior to the meeting.	Missed one to two meetings with valid reasons and informed the team about absence prior to the meeting.	Turned up consistently for meetings. Initiated and set agenda for meetings.
	Frequently late for meetings by more than 30 minutes or left early after a short duration of time meetings.	Frequently late by 5-10 minutes and occasionally late for more than 30 minutes. Occasionally left early.	On time for meetings. Occasionally late for meetings by 5-10 minutes. Stayed for the whole duration of meetings.	Never late for meetings. Stayed for the whole duration of meetings.
	Frequently missed agreed tasks deadlines.	Missed one or two agreed tasks deadlines.	Missed one or two agreed task deadlines but provides valid reasons and notifies the team 1–2 days in advance, including proposing and meeting new deadlines.	Consistently delivers tasks on time and proactively initiates discussions to establish reasonable schedules for the team or themselves.
	Little or no contribution to discussions and activities.	Some contributions to discussions and activities.	Consistently contributed to all discussions and activities.	Led the team in discussions and activities. Willing to follow when others took initiative to lead.
Working with others	Unwilling to interact with others. Inflexible and negative to new ideas suggested.	Usually helpful and receptive towards new ideas suggested by others. At times, insisted on own way of doing things or implemented ideas without the team's consensus.	Helpful and receptive towards new ideas suggested by others. A pleasure to work with.	Expressed ideas clearly, honestly and with respect for others and their work. Willing to listen without arguing every point.
	Never bothered to contribute any ideas to the team.	Contributed some good ideas to the team.	Constantly suggested new ideas to the team. Not all the ideas were applicable though.	Consistently suggested new ideas or was able to build on ideas suggested by others and all of them were good or creative.
Quantity of work	Did less than their fair share of load.	Contributed a fair share of the team's load. However, did more of a specific type of task only (e.g. coding)	Contributed a fair share of the team's load in all aspects.	Contributed a fair share of the team's load in all aspects. Enthusiastically took on responsibilities in various tasks areas. Coached others in the team about how to get their work done.
Quality of work	Work often had flaws and omissions that needed fixing.	Acceptable quality of work that needed frequent minor fixes or the occasional rework.	Good quality of work with occasional minor fixes.	High quality of work that went well beyond expectations.

Professionalism

- All submissions are to be done via **eLearn > Assignments**. All submissions via email will be ignored.

- The zip file should contain the application source code and presentation slides.

- The application source code should include following folders and files:
  - **compile.sh** and **run.sh** batch files
    - The compile.sh file compiles all your Java source files and automatically stores them in the classes directory.
    - The run.sh runs your application.

    Note: You can assume that the instructors' laptops have the PATH environment variable set correctly so that javac.exe and java.exe can be executed at the DOS command prompt from any directory. The instructors will run compile.sh and run.sh on their laptops during assessment.

  - ***src***
    This directory contains all your Java source files.

  - **classes**
    This directory should be left empty during submission. After compile.bat runs, the class files will be stored here automatically.

  - **media files**
    This directory contains any image and audio files used by your application. It will be empty if you build a console application.

  - **lib (if using any external libraries)**
    This directory contains any jar files that you use for your application.

- Alternatively, you can use Maven. If your team chooses to use Maven, you may follow the standard Maven directory structure (src/main/java) instead of the above batch files and folder structure, provided the project compiles and runs successfully.

- A *GX-TY.pptx (or pdf)* of your presentation slides.
  - Include an "AI Use" section listing the AI tools and prompts used for major architectural decisions.
  - If your submission exceeds the eLearn file size limit, you may add an external link to your presentation deck as an appendix.
    - You can hide your appendix slides.

- Only the use of external free or open source libraries is allowed. No other form of code-sharing is allowed. Co-development of code with members from other teams is a definite NO-NO.

Use of AI tools

The use of AI tools (e.g., GitHub Copilot, ChatGPT) is permitted for this project, provided the team maintains full accountability for the submitted work.

- **Every team member must be able to explain the logic and implementation of any AI-assisted code during the presentation.** AI outputs can be inaccurate or outdated; you must manually verify that all code compiles and adheres to the project rules. "AI-generated" is not a valid justification for a logic error or a "hacky" solution.

- To ensure you use these tools as an architect rather than a shortcut, we suggest going through the LearnPrompting.org (Intro &Advanced Prompt Engineering) curriculum.

- All AI usage must be transparently documented:
  - Use comments to attribute specific methods. For example:

    ```
    // Generated by ChatGPT-4; modified for tie-breaker logic
    ```

  - Include an "AI Use" section in your presentation deck's appendix listing the tools and prompts used for major architectural decisions.

- **Note:** AI tools are strictly prohibited during lab tests and exams. **We strongly recommend regular coding practice to ensure individual proficiency for these assessments.**

Project Functional Requirements

Implement the Splendor card game.

1. It should allow 2 or more players. The game should allow human vs computer(s) and human vs human(s).

2. The project is for you to experience working in a team, and to write code that is longer and more complex than your ICEs. The project will not be a differentiator unless a team chooses to neglect it.

3. It can be a networked application (if you wish). Note that it adds complexity and is only worth max 1% of final grade (bonus).

4. The rules of the game:
    a. Splendor Rulebook
    b. How to play Splendor

5. The is the mimum game logic should be implemented:
    a. The game must follow the standard Splendor rules (e.g., 15 prestige points to win).
    b. You implement Gems, Development Cards (Levels 1-3), and Nobles.
    c. The engine must prevent illegal moves (e.g., taking gems that don't exist) to avoid the "usability issues" penalty.
    d. The application must correctly identify the winner and handle tie-breakers as per the rulebook.
    e. The default winning condition is 15 prestige points. However, your design should allow this value to be easily changed via an external configuration file named **config.properties** (e.g., to play a shorter game to 10 points or a longer one to 21).
    f. As noted on page 1, the initial gem counts and paths to card data files should also be in the configuration file.

6. Feel free to make your assumptions for requirements not stated in this document as long as the assumptions do not conflict with the mandatory project requirements. Just document them down.

Project Rubrics

	Evaluation Scale (1 – 20)			
	1 – 8	**9 - 12**	**13 - 16**	**17 - 20**
Code	There was no attempt to modularize the code with multiple classes and methods.	There was some attempt to modularize the code. A lot of repetitive code.	Most of the code was modularized with the exception of a few 'god' classes.	Applied good object modularization, methods perform one function only.
	There are multiple coding styles within the program. It does not adhere to the Java coding convention. There was no use of packages.	The coding style was consistent but it does not adhere to the Java coding convention. There are Java packages but the intent of each package wasn't clear.	Most of the code follows Java coding convention with a few exceptions. Clear use of packages.	The code follows Java coding convention consistently. Clear use of packages.
	All configuration parameters (e.g. database URL) are hard-coded. Magic numbers were used.	Some configuration parameters (e.g. database URL) are hard-coded	All configuration parameters are externalized. There are occasional uses of magic numbers.	All configuration parameters are externalized. Constants are used instead of littering values all over.
	No obvious attempt to design the solution. It's a brute-force solution. All the print statements littered over the place.	Some attempt to do design. There are entity classes found.	Good attempt on doing object design. There was also the idea of separating the model from the view.	The design was clean, and the reader can easily figure out the intent of each class and the relationships between classes. It's a piece of art!
Application	The application is straightforward and is handled with an if-else statement. On top of that, it's buggy.	The application is straightforward and is handled with an if-else statement, or the application has some complex situation but has kinks all over.	The application has some complex scenarios and are reasonably handled. The app is mostly intuitive with some kinks.	Application has a lot of complex scenarios, and they are well-handled (e.g. tie breaker between users, lots of winning conditions).
	The solution does not work or has lots of bugs (e.g. stack trace, crashes).	The solution somewhat works but has usability issues or occasional bugs.	The solution works with some usability or occasional bugs.	The app is intuitive (e.g. all prompts, alerts, statuses, etc. are well placed). The solution works perfectly.
	The application UI is puzzling and/or hard to navigate.	The application UI is decent with some usability issues or occasional bugs.	The game was fun.	The game is so addictive to play it over and over again.
Presentation	The presentation missed the points required.	The presentation covers 50% of the main points but need not elaborate well.	The presentation covers most or all the main points and has not elaborated all the points fully.	The presentation covers all the main points and elaborates well.
	The presentation deck was unclear, messy and with bad contrast or small fonts.	The presentation deck is mostly decent with occasional flaws (e.g. font, contrast).	The presentation deck is decent. The team used it effectively to explain their project.	The presentation deck was well-put together. Diagrams are clear and illustrate the design.
	The presentation is not well-paced, and is hard to follow.	The presentation has some pacing issues and/or is frequently hard to follow.	The presentation was mostly well-paced. However, the team needs to switch laptops or restart the application once.	The presentation was well-paced.
	The team has not prepared or rehearsed on the demo.	The demo flow was mostly too fast or confusing. There was insufficient time allocated for the demo.	The demo flow was decent with occasional hiccups.	The demo showed the features well.